CL-2005: Database Systems
Lab # 04: SQL

Objective

This lab focuses on SQL concepts including **Aggregate Functions, Group by, String Functions, Date Functions, Mathematical Functions, Conversion Functions, and Views.**

Scope

By the end of this lab, students will be able to:

- Utilize **Aggregate Functions** to perform mathematical operations on datasets.
- Apply aggregate functions with the **GROUP BY** clause to summarize and analyze grouped data.
- Filter aggregated results using the **HAVING** clause.
- Manipulate and extract information using **String Functions**.
- Apply **Date and Time Functions** for efficient time-based analysis.
- Use **Mathematical Functions** for numeric computations.
- Convert data types with **CAST() and CONVERT()**.
- Create, update, and manage **Views** for better data presentation and security.

Discussion

1. Aggregate Functions

Aggregate functions **perform calculations** on multiple rows of data and return a single value.

Function	Description	Example
COUNT()	Returns the number of records	COUNT(*)
SUM()	Returns the sum of values	SUM(salary)
AVG()	Returns the average of values	AVG(salary)
MIN()	Returns the minimum value	MIN(salary)
MAX()	Returns the maximum value	MAX(salary)

Example:

```
SELECT AVG(salary) AS AvgSalary
FROM Employees;
```

This returns the average salary for all the employees.

---

2. String Functions

Used to manipulate character strings in SQL.

Function	Description	Example
LEN()	Returns length of a string	LEN('Hello') → 5
UPPER()	Converts string to uppercase	UPPER('hello') → HELLO
LOWER()	Converts string to lowercase	LOWER('HELLO') → hello
CONCAT()	Joins two or more strings	CONCAT('Hello', ' World') → Hello World
SUBSTRING()	Extracts part of a string	SUBSTRING('Database',1,4) → Data
LEFT()	Extracts a specific number of characters from the left side of a string	LEFT('Employee', 3) → Emp
RIGHT()	Extracts a specific number of characters from the right side of a string	RIGHT('Database', 4) → base
LTRIM()	Removes leading spaces from a string	LTRIM('SQL') → SQL
RTRIM()	Removes trailing spaces from a string	RTRIM('SQL ') → SQL
REPLACE()	Replaces occurrences of a substring	REPLACE ('Hello World', 'World', 'SQL') → Hello SQL
CHARINDEX()	Finds the starting position of a substring within a string	CHARINDEX ('World', 'Hello World') → 7

Example:

```
SELECT CONCAT(first_name, ' ', last_name) AS FullName FROM Employees;
```

This concatenates first and last names.

---

3. Date Functions

Used to perform operations on date/time values.

Function	Description	Example	Output
GETDATE()	Returns the current date and time	GETDATE()	2025-02-21 12:34:56
DATEADD()	Adds a specified time interval	DATEADD(DAY, 7, GETDATE())	2025-02-28
DATEDIFF()	Returns the difference between two dates	DATEDIFF(YEAR, '2000-01-01', GETDATE())	25
FORMAT()	Formats date/time values	FORMAT(GETDATE(), 'MM-dd-yyyy')	02-21-2025
EOMONTH()	Returns the last day of a given month	EOMONTH(GETDATE())	2025-02-28
DATEPART()	Extracts a specific part of a date	DATEPART(YEAR, GETDATE())	2025

Example:

```
SELECT GETDATE() AS CurrentDate;
```

This returns the current date and time.

---

4. Mathematical Functions

Mathematical functions are essential for **data analysis and computations**. They help in:

- Performing **rounding operations** for better financial reports.
- Computing absolute values for **error handling**.
- Rounding up or down numbers for **data precision**.

Mathematical Functions in SQL

Function	Description	Example	Output
ROUND()	Rounds a number to a specified decimal place	ROUND(123.456, 2)	123.46
ABS()	Returns absolute value	ABS(-10)	10
FLOOR()	Rounds down to the nearest integer	FLOOR(5.9)	5
CEILING()	Rounds up to the nearest integer	CEILING(5.1)	6
POWER()	Raises a number to a power	POWER(2, 3)	8
SQRT()	Returns the square root of a number	SQRT(25)	5

RAND()	Generates a random number between 0 and 1	RAND()	0.45678
SIGN()	Returns the sign of a number (-1, 0, 1)	SIGN(-50)	-1
EXP()	Returns the exponent of a number	EXP(2)	7.3891
LOG()	Returns the natural logarithm of a number	LOG(10)	2.3025
LOG10()	Returns the base-10 logarithm of a number	LOG10(1000)	3
PI()	Returns the value of PI	PI()	3.1416
DEGREES()	Converts radians to degrees	DEGREES(PI())	180
RADIANS()	Converts degrees to radians	RADIANS(180)	3.1416

Example:

```
SELECT ROUND(salary, 2) AS RoundedSalary, ABS(-salary) AS AbsoluteSalary
FROM Employees;
```

This retrieves salary rounded to two decimal places and converts negative salaries into positive values.

---

5. Conversion Functions

Conversion functions in SQL allow changing the **data type** of a value from one format to another. This is particularly useful when dealing with numeric, string, and date-based transformations. Some common scenarios include:

- Converting a floating-point number to an integer.
- Changing a date format to a readable string.
- Ensuring data type consistency during queries.
- Handling errors in type conversion by returning NULL for invalid conversions.

Function	Description	Example	Output
CAST()	Converts a value from one data type to another	CAST(123.45 AS INT)	123
CONVERT()	Converts a value and formats dates	CONVERT(VARCHAR, GETDATE(), 103)	21/02/2025
TRY_CAST()	Attempts to cast a value and returns NULL if unsuccessful	TRY_CAST('abc' AS INT)	NULL

TRY_CONVERT()	Attempts to convert a value and returns NULL if unsuccessful	TRY_CONVERT(INT, 'xyz')	NULL
PARSE()	Converts a string into a specified data type, mainly for dates	PARSE('Feb 21, 2025' AS DATE)	2025-02-21
FORMAT()	Formats a value with a specified format string	FORMAT(GETDATE(), 'MM-dd-yyyy')	02-21-2025

Key Differences Between CAST() and CONVERT()

- **CAST()** follows standard SQL syntax and is preferred for portability.
- **CONVERT()** provides additional formatting options, especially for dates.
- **TRY_CAST()** and **TRY_CONVERT()** handle conversion errors by returning NULL instead of an error.
- **PARSE()** is mainly used for **converting strings into dates and numbers**, and it requires a valid culture format.
- **FORMAT()** is ideal for **custom formatting** of dates and numbers.

Here's how different **style codes** work with the `CONVERT()` function:

Style Code	Format	Example Output
101	mm/dd/yyyy	02/21/2025
102	yyyy.mm.dd	2025.02.21
103	dd/mm/yyyy	21/02/2025
104	dd.mm.yyyy	21.02.2025
105	dd-mm-yyyy	21-02-2025
106	dd Mon yyyy	21 Feb 2025
107	Mon dd, yyyy	Feb 21, 2025
110	mm-dd-yyyy	02-21-2025
111	yyyy/mm/dd	2025/02/21
112	yyyymmdd	20250221

---

6. Views

Views **simplify and secure** data representation. They are useful for:

- **Encapsulation**: Hides complex joins and computations.
- **Security**: Limits user access to specific columns.
- **Performance Optimization**: Stores frequent queries for efficiency.

Creating a View

```
CREATE VIEW EmployeeView AS SELECT first_name, last_name, department
FROM Employees;
```

Updating a View

```
ALTER VIEW EmployeeView AS
SELECT first_name, last_name, salary FROM Employees;
```

Dropping a View

```
DROP VIEW EmployeeView;
```

---

7. Group By Statement

The Group By statement groups rows that have same values into summary rows.

It is often used with aggregate functions to group the result set by one or more columns.

The aggregate functions include:

- COUNT(column_name) – Returns the number of rows in a group, ignoring NULLs.
- MIN(column_name) – Returns the smallest value in a column.
- MAX(column_name) – Returns the largest value in a column.
- SUM(column_name) – Returns the total sum of numeric values in a column.
- AVG(column_name) – Returns the average of numeric values in a column.
- STDEV(column_name) – Returns the standard deviation of values in a column.
- VAR(column_name) – Returns the variance of values in a column.
- STRING_AGG(column_name, delimiter) – Concatenates string values into a single string, separated by a specified delimiter.

You can find further documentation on aggregate functions on: https://learn.microsoft.com/en-us/sql/tsql/functions/aggregate-functions-transact-sql?view=sql-server-ver16

- **Command**:

```
SELECT column_name(s), AGGREGATE_FUNCTION(column_name)
FROM table_name
GROUP BY column_name(s)
```

- **Example**:

```
SELECT mode, SUM(amount) AS total
FROM payment
GROUP BY mode;
```

GROUP BY with Multiple Columns

```
SELECT department, job_title, COUNT(*) AS
EmployeeCount FROM Employees
GROUP BY department, job_title;
```

This groups employees by department and job title.

---

8. HAVING Clause

The **HAVING** clause is used to apply a filter on the result of **GROUP BY** based on the specified condition.

The **WHERE** clause places conditions on the **selected** columns, whereas the **HAVING** clause places conditions on groups created by **GROUP BY** clause.

- **Command**:

```
SELECT column_name(s)
FROM table name
WHERE condition(s)
GROUP BY column_name(s)
HAVING condition(s)
```

- **Example**

```
SELECT mode, COUNT(amount) AS total
FROM payment
GROUP BY mode
HAVING COUNT(amount)>=3
ORDER BY total ASC;
```

Practice Queries

1. Retrieve the max and min salaries.
2. Extract first three characters of employee names.
3. Find the difference in days between hire date and current date.
4. Convert current date into mm-dd-yyyy format.
5. Create a view that selects employees from 'HR' department.
6. Write a query using ROUND() to round salary values to the nearest integer.
7. Use EOMONTH() to find the last date of the current month.
8. Write a query using POWER() to calculate 5 raised to the power of 3.
9. Convert the string '100.25' into an integer using CAST() and CONVERT().

Solution

```
-- Step 1: Create Database
CREATE DATABASE EmployeeDB;

-- Step 2: Use the created database
USE EmployeeDB;

-- Step 3: Create Employees Table
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(100),
    DepartmentID INT,
    Department VARCHAR(50),
```

```
    Salary DECIMAL(10,2),
    HireDate DATE
);

-- Step 4: Insert Sample Data
INSERT INTO Employees (EmployeeID, EmployeeName, DepartmentID, Department, Salary,
HireDate) VALUES (1, 'John Doe', 101, 'HR', 55000.75, '2018-06-15'),
(2, 'Jane Smith', 102, 'Finance', 72000.50, '2016-03-10'),
(3, 'Mike Johnson', 103, 'IT', 85000.00, '2020-11-20'),
(4, 'Emily Davis', 101, 'HR', 62000.25, '2017-09-05'),
(5, 'Robert Brown', 102, 'Finance', 67000.80, '2019-07-22'),
(6, 'Lisa Wilson', 103, 'IT', 90000.60, '2015-01-18');

-- Query 1: Retrieve max and min salaries
SELECT MAX(Salary) AS MaxSalary, MIN(Salary) AS MinSalary
FROM Employees;

-- Query 2: Extract the first three characters of employee names
SELECT EmployeeID, EmployeeName, LEFT(EmployeeName, 3) AS
NamePrefix FROM Employees;

-- Query 3: Find the difference in days between hire date and current date
SELECT EmployeeID, EmployeeName, DATEDIFF(DAY, HireDate, GETDATE()) AS
DaysSinceHire FROM Employees;

-- Query 4: Convert current date into mm-dd-yyyy format
SELECT FORMAT(GETDATE(), 'MM-dd-yyyy') AS FormattedDate;

-- Query 5: Create a view that selects employees from 'HR' department
CREATE VIEW HR_Employees
AS SELECT * FROM
Employees
```

```
WHERE Department = 'HR';

-- Retrieve data from the HR_Employees view
SELECT * FROM HR_Employees;

-- Query 6: Round salary values to the nearest integer
SELECT EmployeeID, EmployeeName, Salary, ROUND(Salary, 0) AS
RoundedSalary FROM Employees;

-- Query 7: Find the last date of the current month using EOMONTH()
SELECT EOMONTH(GETDATE()) AS LastDayOfMonth;

-- Query 8: Calculate 5 raised to the power of 3 using POWER()
SELECT POWER(5, 3) AS PowerResult;

-- Query 9: Convert the string '100.25' into an integer using CAST() and CONVERT()
SELECT
   CAST('100.25' AS INT) AS CastResult,
   CONVERT(INT, '100.25') AS ConvertResult;
```
